CPM · ROS2_WRAPPER ADAPTER LAYER

The ROS2 Wrapper Layer — How It Actually Works

RosInputInterface<T> / RosOutputInterface<T> — structure, data flow, threading — worked through with a real channel

01 THE PROBLEM IT EXISTS TO SOLVE

Every real call site in `LpsSaJobMgrScs.cpp` / `LpsSaScs.cpp` was written against SCS's synchronous, poll-based API:

SCS (ORIGINAL)

```
bool InputInterface<T>::get(T& data);
bool OutputInterface<T>::publish(const T& data);
```

ROS2/DDS (NATIVE)

```
rclcpp::Subscription<T>  // cal
rclcpp::Publisher<T>     // no
```

DDS has no `get()` — it delivers messages by firing a callback whenever data arrives. Rewriting every call site to that style risks introducing logic bugs in business code that doesn't otherwise need to change. The wrapper absorbs the mismatch in one shared place instead: it presents SCS's old interface on the outside, DDS's real mechanism on the inside.

02 THE TWO CLASSES

Both live in `CPM-Loader-ais/prod/common/ros2_wrapper/` , header-only templates, one instantiation per real message type:

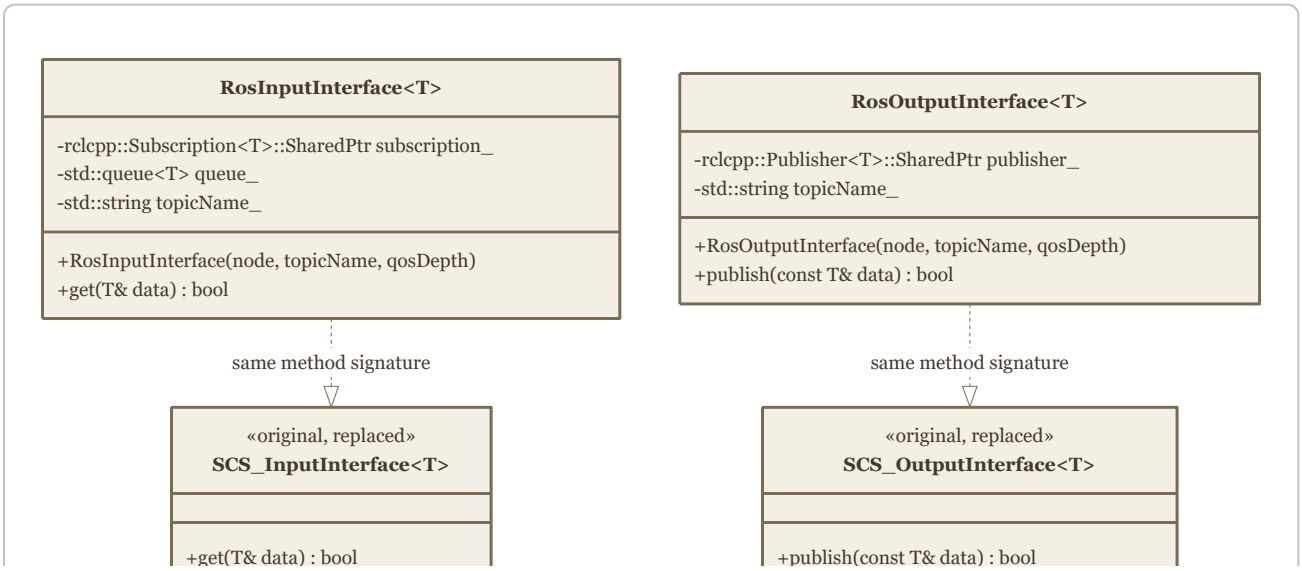


FIG 1 — BOTH WRAPPERS MATCH THEIR SCS COUNTERPART'S PUBLIC METHOD EXACTLY; ONLY THE DECLARED MEMBER TYPE AT EACH CALL SITE CHANGES

---

03 INPUT SIDE, IN DETAIL — A REAL EXAMPLE

Using leg 1 (`LpsSaWeighReqstChannel`) concretely: JobMgr sends a request, WeighApp's `LpsSaWeighScsReqstIn` receives it and `LpsSaScsChkForReqst()` drains it.

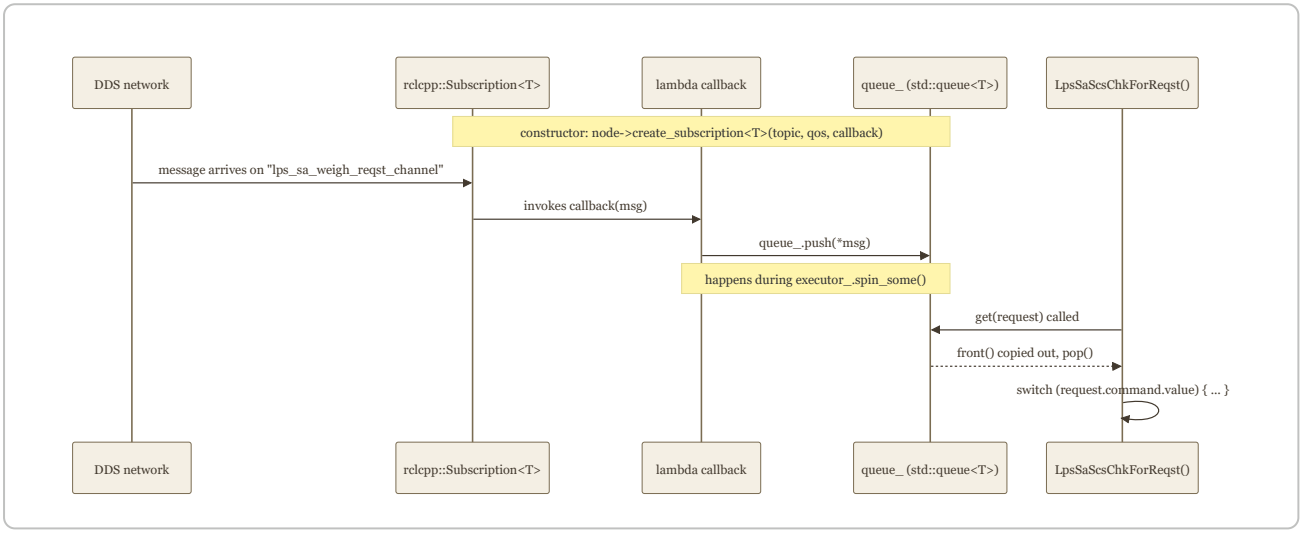


FIG 2 — ONE MESSAGE'S PATH FROM THE NETWORK TO BUSINESS LOGIC

The constructor does the actual DDS work — one line, everything else is bookkeeping:

```
RosInputInterface(const rclcpp::Node::SharedPtr& node, const std::string& topicName
    : topicName_(topicName)
{
    subscription_ = node->create_subscription<T>(
        topicName, qosDepth,
        [this](const typename T::SharedPtr msg) { queue_.push(*msg); }
    );
}

bool get(T& data) {
    if (queue_.empty()) return false;
    data = queue_.front();
    queue_.pop();
    return true;
}
```

Why a real queue, not a cached latest value: real SCS code drains with `while (channel->get(x)) { … }` , expecting every message since the last poll — a single cached

value would silently drop a message if two arrived between polls.

04 OUTPUT SIDE, IN DETAIL — THE SAME LEG'S OTHER DIRECTION

On JobMgr's side, `weighAppInf_` (a `LpsSaWeighAppInf` instance) holds `requestOutput_` and calls `sendRequest()`:

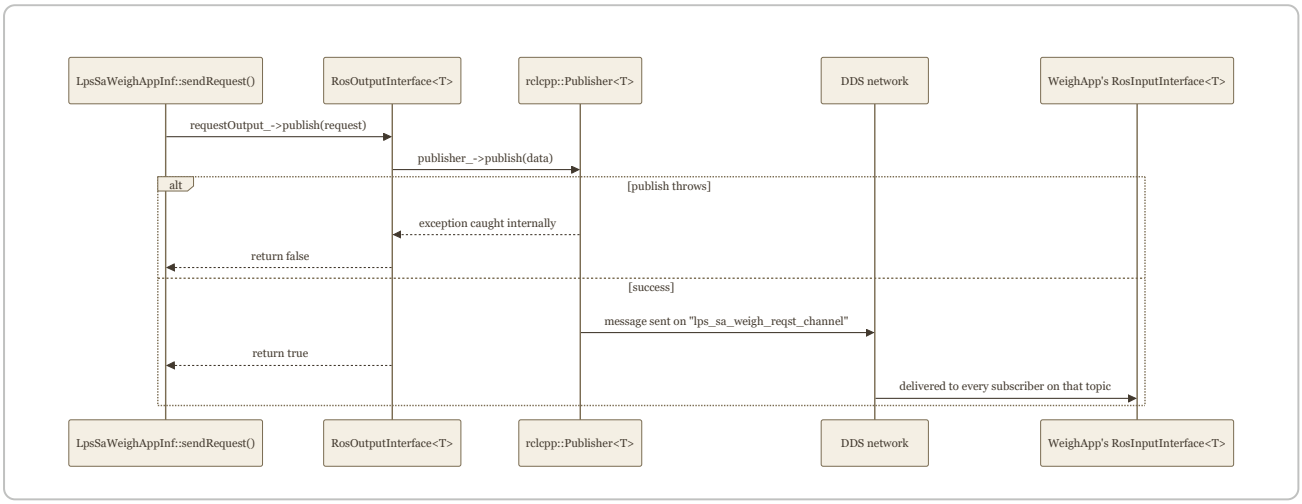
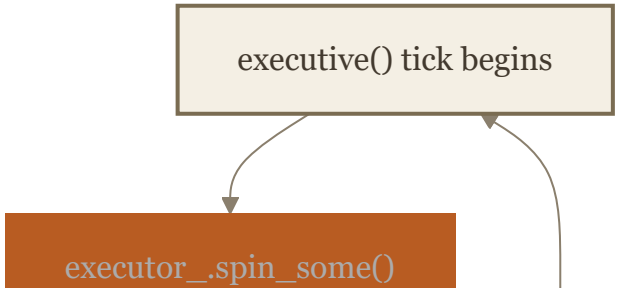


FIG 3 — PUBLISH() MIRRORS THE REAL SCS CONTRACT: CALLERS CHECK THE BOOL RETURN, NO EXCEPTION EVER ESCAPES

```
bool publish(const T& data) {
    try {
        publisher_->publish(data);
        return true;
    } catch (const std::exception&) {
        return false;   // SCS callers check the return value, not a catch block
    }
}
```

05 THE THREADING MODEL — WHY THERE'S NO MUTEX

Both classes are deliberately not thread-safe on their own — and that's fine, because of how `executive()` is structured. Every real app's tick looks like this:



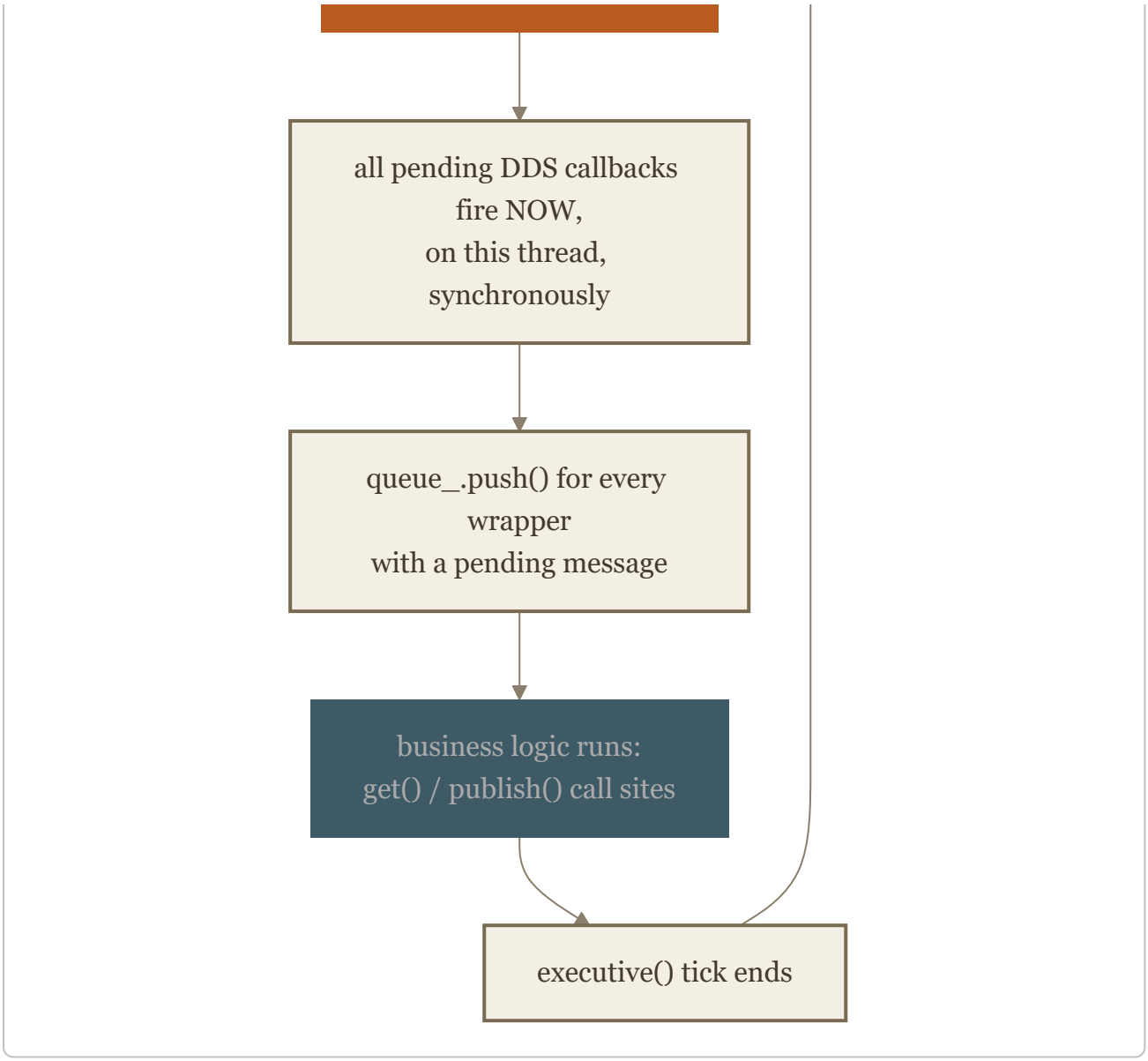


FIG 4 — ONE TICK, SINGLE THREAD, START TO FINISH

`spin_some()` (which runs the subscription callbacks, i.e. every `queue_.push()`) and `get()` / `publish()` (business logic) run on the **same thread**, and `spin_some()` always finishes completely before business logic starts, every tick. There's no window where one could interrupt the other — so no mutex is needed, unlike a background-thread-spin design, which would.

THE ONE REQUIRED ADDITION PER APP

`executor_.spin_some();` — one line, first thing in `executive()`, before any wrapper's `get()` / `publish()` is called that tick. Miss this and messages queue up in DDS but never actually arrive in any `queue_`.

06 PUTTING IT TOGETHER — ONE FULL ROUND TRIP

Leg 1 and leg 2 chained across both apps, both ticks, so the whole shape is visible at once:

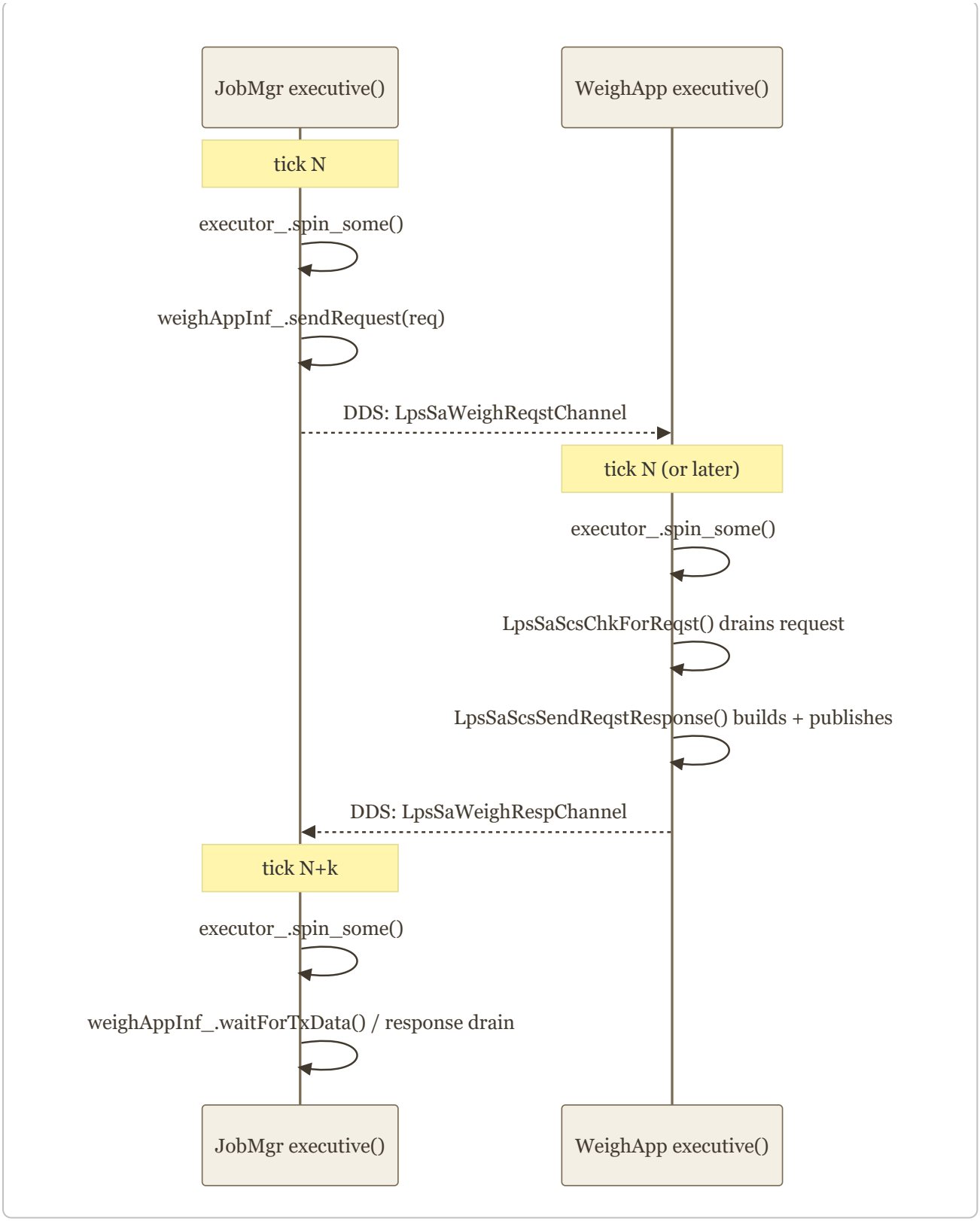


FIG 5 — THE TWO APPS AREN'T SYNCHRONIZED TICK-FOR-TICK; DDS JUST DELIVERS WHENEVER THE NEXT SPIN_SOME() HAPPENS TO RUN

Nothing here assumes the two apps' ticks line up — that's the point of using a real queue and a real pub/sub transport instead of a tighter coupling: whichever tick happens to run `spin_some()` after a message was sent is the one that picks it up.

CPM-Loader-ais/prod/common/ros2_wrapper/{RosInputInterface.h,RosOutputInterface.h} · worked example: leg 1/2 (LpsSaWeighReqstChannel/LpsSaWeighRespChannel)